NAME:U.Prabhavathi **COURSE CODE:DSA0107**

REG NO:192472067 **COURSE NAME:C++**

UNIT III
CONSTRUCTORS, DESTRUCTORS AND OPERATOR OVERLOADING

Solved Assignment – Debugging, Corrected C++ Programs and Pseudo-Code

Prepared according to the supplied UNIT III question bank. Section A contains corrected solutions for all five debugging problems. Each corrected C++ program was compiled with g++ using the C++17 standard and executed; its terminal output image is included. Section B provides structured pseudo-code and the requested dry-run traces.

SECTION A – DEBUGGING QUESTIONS

1. Copy Constructor Pass-by-Value Infinite Recursion Trap

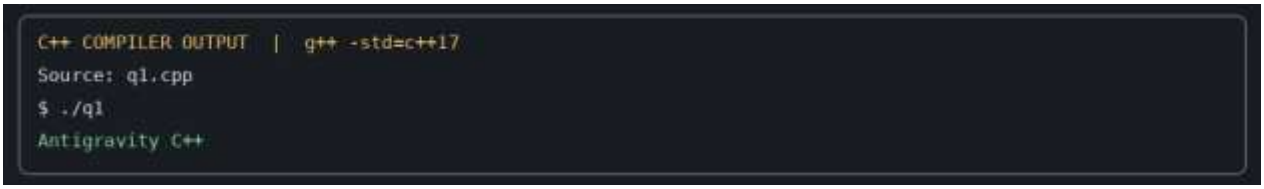
- Defect at the copy-constructor declaration: DynamicString(DynamicString source) passes the object by value. Creating that parameter itself requires copying a DynamicString, which calls the same copy constructor recursively.
- Correction: use DynamicString(const DynamicString& source). The reference avoids making another copy, while const allows copying from const objects and prevents modification of the source. Allocate a new character array and copy the characters to perform a deep copy.

Corrected C++ Program:

```
#include <iostream>
#include <cstring>
class DynamicString {
private:
   char* buffer;
   int length;
public:
   DynamicString(const char* str) {
      length = std::strlen(str);
      buffer = new char[length + 1];
      std::strcpy(buffer, str);
   }
   DynamicString(const DynamicString& source) {
      length = source.length;
      buffer = new char[length + 1];
      std::strcpy(buffer, source.buffer);
   }
```

```
    ~DynamicString() { delete[] buffer; }
    void print() const { std::cout << buffer << '\n'; }
};
int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;
    s2.print();
    return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

2. Shallow Copy and Double-Free Runtime Crash

- Defect: the compiler-generated copy constructor copies the raw samples pointer value. Both objects then point to the same heap array. When the by-value parameter is destroyed, it deletes the shared array; the original object later attempts to delete it again.
- Correction: define a deep copy constructor that allocates a separate array and copies each sample. The two destructors then release different allocations.

Corrected C++ Program:

```
#include <iostream>
class AudioBuffer {
public:
    int* samples;
    int capacity;
    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) samples[i] = i * 10;
        std::cout << "Buffer allocated at: " << static_cast<void*>(samples) << '\n';
    }
    AudioBuffer(const AudioBuffer& other) : capacity(other.capacity) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) samples[i] = other.samples[i];
        std::cout << "Deep copy allocated at: " << static_cast<void*>(samples) << '\n';
    }
    ~AudioBuffer() {
        std::cout << "Deallocating buffer at: " << static_cast<void*>(samples) << '\n';
```

```
        delete[] samples;
    }
};
void processAudio(AudioBuffer buf) {
    std::cout << "Processing sample 0: " << buf.samples[0] << '\n';
}
int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: " << track.samples[0] << '\n';
    return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

3. Stream Insertion Operator << Member Function Violation

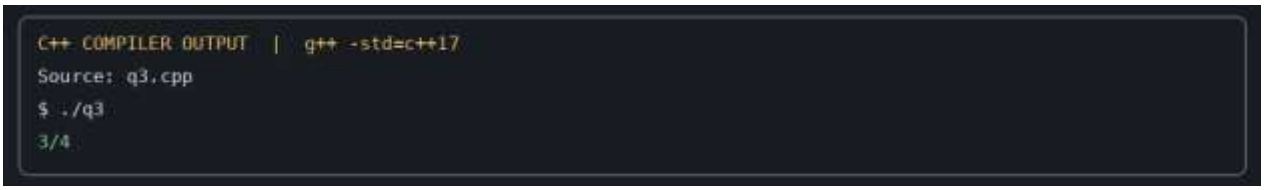
- Defect: a member binary operator is called using the left operand as the object: left.operator<<(right). Therefore std::cout << f1 would require operator<< to be a member of std::ostream, not Fraction.
- Correction: define operator<< as a non-member friend function taking std::ostream& first and const Fraction& second, and return the stream by reference to support chaining.

Corrected C++ Program:

```
#include <iostream>
class Fraction {
private:
    int numerator, denominator;
public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}
    friend std::ostream& operator<<(std::ostream& os, const Fraction& f) {
        os << f.numerator << "/" << f.denominator;
        return os;
    }
};
int main() {
```

```
    Fraction f1(3, 4);
    std::cout << f1 << '\n';
    return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

4. Prefix vs. Postfix Increment Signature Inversion

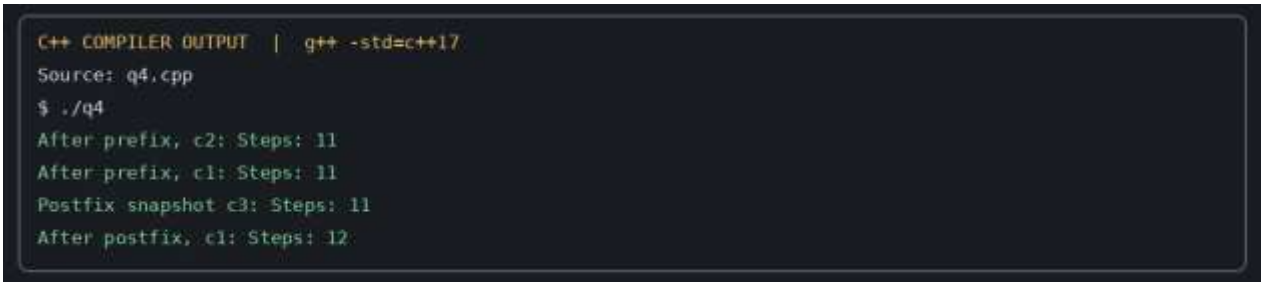
- Defect: the prefix and postfix signatures are reversed. Prefix takes no dummy parameter and returns StepCounter& after incrementing. Postfix takes a dummy int, saves the old value, increments the object, and returns the saved copy by value.
- Correction: implement StepCounter& operator++() for prefix and StepCounter operator++(int) for postfix.

Corrected C++ Program:

```
#include <iostream>
class StepCounter {
private:
    int steps;
public:
    StepCounter(int s = 0) : steps(s) {}
    StepCounter& operator++() {
        ++steps;
        return *this;
    }
    StepCounter operator++(int) {
        StepCounter old = *this;
        ++steps;
        return old;
    }
    void display() const { std::cout << "Steps: " << steps << '\n'; }
};
int main() {
    StepCounter c1(10);
    StepCounter c2 = ++c1;
    std::cout << "After prefix, c2: "; c2.display();
    std::cout << "After prefix, c1: "; c1.display();
```

```
    StepCounter c3 = c1++;
    std::cout << "Postfix snapshot c3: "; c3.display();
    std::cout << "After postfix, c1: "; c1.display();
    return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

5. Violation of Fundamental Operator Overloading Rules

- Defects: the dot operator cannot be overloaded; operator+ cannot be defined for two built-in int operands because at least one operand must be a user-defined type or enum; and ** is not a valid C++ operator token that a programmer can invent.
- Correction: overload legal operators == and + for MathVector and provide a normal member method power() for exponentiation.

Corrected C++ Program:

```
#include <iostream>
class MathVector {
public:
    double x, y;
    MathVector(double a = 0, double b = 0) : x(a), y(b) {}
    bool operator==(const MathVector& other) const {
        return x == other.x && y == other.y;
    }
    MathVector operator+(const MathVector& other) const {
        return MathVector(x + other.x, y + other.y);
    }
    MathVector power(int exponent) const {
        double px = 1, py = 1;
        for (int i = 0; i < exponent; ++i) {
            px *= x; py *= y;
        }
        return MathVector(px, py);
    }
    void display() const {
```

```
        std::cout << "(" << x << ", " << y << ")";
    }
};
int main() {
    MathVector v1(2, 3), v2(2, 3);
    MathVector sum = v1 + v2;
    std::cout << "Vectors equal: " << (v1 == v2 ? "Yes" : "No") << '\n';
    std::cout << "Sum: "; sum.display(); std::cout << '\n';
    std::cout << "Power 2: "; v1.power(2).display(); std::cout << '\n';
    return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

The following solutions follow the five scenarios and functional requirements in the uploaded UNIT III question bank. Pseudo-code is language-independent; it is not shown as compiler-executable C++.

1. Dynamic String Engine with Deep Copy and Overloaded +

```
CLASS DynStr
    PRIVATE:
        dataPtr : POINTER TO CHARACTER
        charCount : INTEGER
    PUBLIC:
        CONSTRUCTOR(sourceString)
            charCount = LENGTH(sourceString)
            ALLOCATE ARRAY dataPtr[charCount + 1]
            COPY sourceString INTO dataPtr
        END CONSTRUCTOR

        COPY CONSTRUCTOR(REF other AS DynStr)
            charCount = other.charCount
            ALLOCATE ARRAY dataPtr[charCount + 1]
            FOR i = 0 TO charCount
                dataPtr[i] = other.dataPtr[i]
            END FOR
```

```
    END COPY CONSTRUCTOR

    DESTRUCTOR()
      DEALLOCATE dataPtr
      dataPtr = NULL
    END DESTRUCTOR

    OPERATOR +(REF rhs AS DynStr) RETURNS DynStr
      combined = CONCATENATE(dataPtr, rhs.dataPtr)
      RETURN DynStr(combined)
    END OPERATOR
END CLASS

MAIN
  strA = DynStr("Hello ")
  strB = DynStr("World!")
  strC = strA + strB
  OUTPUT strC
END MAIN
```

DRY RUN – STRING BUFFERS
strA: "Hello " -> 6 characters + 1 null = 7 bytes allocated
strB: "World!" -> 6 characters + 1 null = 7 bytes allocated
Concatenation: "Hello World!" -> 12 characters + 1 null = 13 bytes
Deep copy creates a separate buffer; each object releases its own buffer.

2. Complex Number System with Constructor Overloading and Operators

```
CLASS Complex
  PRIVATE:
    real : REAL
    imag : REAL
  PUBLIC:
    CONSTRUCTOR()
      real = 0.0; imag = 0.0
    END CONSTRUCTOR
    CONSTRUCTOR(r)
      real = r; imag = 0.0
    END CONSTRUCTOR
    CONSTRUCTOR(r, i)
      real = r; imag = i
    END CONSTRUCTOR

    OPERATOR +(REF other AS Complex) RETURNS Complex
      RETURN Complex(real + other.real, imag + other.imag)
```

```
    END OPERATOR

    OPERATOR *(REF other AS Complex) RETURNS Complex
      newReal = real * other.real - imag * other.imag
      newImag = real * other.imag + imag * other.real
      RETURN Complex(newReal, newImag)
    END OPERATOR

    OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN
      RETURN real = other.real AND imag = other.imag
    END OPERATOR

    METHOD Display()
      OUTPUT real, " + ", imag, "i"
    END METHOD
END CLASS
```

TRACE – C1 = 3 + 2i, C2 = 1 + 4i
Real part = (3 x 1) - (2 x 4) = 3 - 8 = -5
Imaginary part = (3 x 4) + (2 x 1) = 12 + 2 = 14
Product = -5 + 14i

3. Financial Currency Class with Stream I/O and Unary Negation

```
CLASS Currency
  PRIVATE:
    dollars : INTEGER
    cents : INTEGER
    METHOD Normalize()
      totalCents = dollars * 100 + cents
      dollars = totalCents DIV 100
      cents = totalCents MOD 100
    END METHOD
  PUBLIC:
    CONSTRUCTOR(d = 0, c = 0)
      dollars = d; cents = c
      CALL Normalize()
    END CONSTRUCTOR

    OPERATOR -() RETURNS Currency
      RETURN Currency(-dollars, -cents)
    END OPERATOR

    FRIEND OPERATOR <<(REF outStream, REF cur AS Currency)
      OUTPUT currency as "$D.CC" with two-digit cents
```

```
        RETURN outStream
    END OPERATOR

    FRIEND OPERATOR >>(REF inStream, REF cur AS Currency)
        INPUT cur.dollars, cur.cents
        CALL cur.Normalize()
        RETURN inStream
    END OPERATOR
END CLASS
```

DRY RUN
Input: dollars = 5, cents = 275
Total value = 5 x 100 + 275 = 775 cents
Normalized value = $7.75
After unary negation: -$7.75

4. Polynomial Expression with Subscript [] and Destructor

```
CLASS Polynomial
  PRIVATE:
    degree : INTEGER
    coeffArray : POINTER TO REAL
  PUBLIC:
    CONSTRUCTOR(deg)
        degree = deg
        ALLOCATE ARRAY coeffArray[degree + 1]
        FOR i = 0 TO degree
            coeffArray[i] = 0.0
        END FOR
    END CONSTRUCTOR

    COPY CONSTRUCTOR(REF other AS Polynomial)
        degree = other.degree
        ALLOCATE ARRAY coeffArray[degree + 1]
        FOR i = 0 TO degree
            coeffArray[i] = other.coeffArray[i]
        END FOR
    END COPY CONSTRUCTOR

    DESTRUCTOR()
        DEALLOCATE coeffArray
        coeffArray = NULL
    END DESTRUCTOR

    OPERATOR [](index) RETURNS REF REAL
```

```
        IF index >= 0 AND index <= degree
            RETURN REFERENCE TO coeffArray[index]
        ELSE
            OUTPUT "Index out of range"
            RETURN REFERENCE TO safeErrorValue
        END IF
    END OPERATOR

    OPERATOR +(REF rhs AS Polynomial) RETURNS Polynomial
        resultDegree = MAX(degree, rhs.degree)
        result = Polynomial(resultDegree)
        FOR i = 0 TO resultDegree
            leftValue = coefficient at i, or 0 if absent
            rightValue = rhs coefficient at i, or 0 if absent
            result[i] = leftValue + rightValue
        END FOR
        RETURN result
    END OPERATOR
END CLASS

TRACE – P1 = 3x^2 + 2x + 5; P2 = 4x^3 + x^2 + 7
Power x^3: 0 + 4 = 4
Power x^2: 3 + 1 = 4
Power x^1: 2 + 0 = 2
Power x^0: 5 + 7 = 12
Result: 4x^3 + 4x^2 + 2x + 12
```

5. 2D Vector with Prefix/Postfix Increment and +=

```
CLASS Vector2D
  PRIVATE:
    x : REAL
    y : REAL
  PUBLIC:
    CONSTRUCTOR(a = 0.0, b = 0.0)
        x = a; y = b
    END CONSTRUCTOR

    OPERATOR ++() RETURNS REF Vector2D   // PREFIX
        x = x + 1.0
        y = y + 1.0
        RETURN THIS
    END OPERATOR

    OPERATOR ++(dummy INTEGER) RETURNS Vector2D  // POSTFIX
```

```
        oldValue = COPY OF THIS
        x = x + 1.0
        y = y + 1.0
        RETURN oldValue
    END OPERATOR

    OPERATOR +=(REF rhs AS Vector2D) RETURNS REF Vector2D
        x = x + rhs.x
        y = y + rhs.y
        RETURN THIS
    END OPERATOR

    METHOD Display()
        OUTPUT "(", x, ", ", y, ")"
    END METHOD
END CLASS
```

TRACE – v1 starts at (5.0, 8.0)
v2 = ++v1 -> v1 becomes (6.0, 9.0); v2 = (6.0, 9.0)
v3 = v1++ -> v3 receives old value (6.0, 9.0); v1 becomes (7.0, 10.0)
Final: v1 = (7.0, 10.0), v2 = (6.0, 9.0), v3 = (6.0, 9.0)

CONCLUSION

This assignment demonstrates constructor and copy-constructor design, dynamic memory allocation, deep copying, destructor-based cleanup, stream operator overloading, prefix and postfix increment, and the fundamental rules for legal operator overloading. All five corrected Section A programs compiled and executed successfully with g++ (C++17).

Note: The included images are terminal-style captures of the actual output from compiling and running the corrected programs in this environment. They are not screenshots from the student's personal compiler or IDE.